STAC AI BOOTCAMP NYC | 21 OCTOBER 2026



Building Trust in Agentic AI: From Evidence to Action

Matthew Dixon, PhD, FRM
BeliefLens

matthew.dixon@belieflens.org

Bootcamp agenda · New York time

Time	Session
9:30–10:15	Financial language-model architectures
10:15–11:15	Building grounded financial RAG
11:15–11:30	Break
11:30–12:30	Post-training for financial applications
12:30–1:15	Lunch
1:15–2:15	Hands-on adaptation lab
2:15–3:00	LLMs and SLMs for alpha and risk
3:00–3:15	Break
3:15–4:15	Evaluation and evidence attribution
4:15–5:00	Deployment and governance clinic

Is AI fooled by randomness?



$$
\begin{array}{cccccccccc}
+1 & -1 & +1 & -1 & -1 & -1 & +1 & -1 & +1 & -1 \\
-1 & -1 & +1 & +1 & +1 & +1 & +1 & -1 & +1 & +1 \\
+1 & -1 & -1 & +1 & -1 & -1 & +1 & +1 & -1 & +1 \\
-1 & +1 & +1 & +1 & -1 & -1 & -1 & -1 & +1 & -1 \\
-1 & -1 & -1 & +1 & -1 & +1 & +1 & +1 & -1 & +1
\end{array}
$$

$+1$: green $\quad -1$: red

Question: What, if anything, can you infer about the next value?

Which sequences show significant autocorrelation?

TWO-SIDED PERMUTATION TEST · 4,000 SHUFFLES · MODEL DOES NOT SEE THESE RESULTS

Sequence	Lag-1 r	Lag-1 p	Max lags 1–5 p	At 5%	At 1%
1: random draws	-0.062	0.720	0.829	No rejection	No rejection
2: random draws	$+0.387$	0.013	0.045	Reject	No rejection
3: random draws	-0.185	0.208	0.502	No rejection	No rejection
4: alternating	-1.000	0.00025	0.00025	Reject	Reject

Sequence 4: $-1, +1, -1, +1, \ldots$; 50 values, ending $+1$. Next under this rule: -1.

r: sample correlation between neighbouring values (lag 1). p: permutation-test tail probability under the exchangeable-order null.

5% significance: reject when $p \leq 0.05$. 1% significance: reject when $p \leq 0.01$.

No rejection means no significant dependence detected by this test. It does not prove randomness or independence.

A blind prompt; a separate statistical check

Exact question after the 50 numbers

Predict the next number in the sequence. Respond only with +1 or -1.

Forced sign only. No generator disclosed. No test results shown.

Compute lag-1 Pearson correlation on adjacent values. Shuffle the whole sequence 4,000 times and check how often the shuffled correlation is at least as strong as the observed correlation.

For lags 1–5, use the maximum absolute correlation in each shuffle.

Why not the ordinary Pearson p-value? Binary observations and overlapping pairs do not match its standard independent-normal-pairs assumptions.

Null: all arrangements of the observed values are equally plausible.

First, let the model explain its answer

Original open-ended prompt after the 50 numbers

What, if anything, can you infer about the next value in this sequence? Explain your reasoning.

Input	Qwen3.5-9B	Llama-3.3-70B	GPT-OSS-120B
Random sequence 1	Indeterminate; 50/50 if fair	Tentative $+1$; states 52%	Indeterminate; 50/50 if fair
Random sequence 2	Roughly 50/50; no consistent rule	Tentative $+1$; run might continue	No reliable prediction
Random sequence 3	No certainty; states 50/50	Tentative -1; supposed grouping	Indeterminate; a specific sign is a guess

Next, require a single answer: $+1$ or -1. Which does the model choose?

Forced prediction: what did the models choose?

Exact instruction after the same 50 numbers

Predict the next number in the sequence. Respond only with +1 or -1.

Input	Qwen3.5-9B	Llama-3.3-70B	GPT-OSS-120B
Random sequence 1	+1	+1	−1 / +1
Random sequence 2	+1	+1	−1
Random sequence 3	+1	−1	−1
Alternating control	−1	−1	−1

Two identical-prompt repeats. Only GPT-OSS sequence 1 changed sign.

A forced guess does not establish that a random sequence is predictable.

A strong answer preference is not a reliable forecast

SAME FORCED-PREDICTION PROMPT · LLAMA-3.3-70B

Input	Generated sign	Score for +1	Score for −1
Random sequence 1	+1	99.13%	0.87%
Random sequence 2	+1	85.20%	14.80%
Random sequence 3	−1	6.01%	93.99%
Alternating control	−1	18.24%	81.76%

These scores compare the likelihood of writing the two signs.

For the independent fair generator, the next random draw remains 50/50.

Sequence 1's +1 score was 97.70% on repeat; the sign stayed unchanged.

From a plausible answer to a trustworthy action

If an AI can misread 50 numbers, what must we verify before it acts on a financial decision?

- **Evidence:** Did it use the right facts, dates and sources?
- **Reliability:** Does the answer survive independent checks and controlled tests?
- **Authority:** Is the proposed action permitted, with an accountable owner?

Python notebooks: the default for every exercise

Open the supplied `STAC_AI_Bootcamp` folder in Terminal. Keep the notebooks, helper files and data together.

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
python -m jupyterlab notebooks
```

Start with `01_architectures.ipynb`. Read the guidance, then run cells from top to bottom with **Shift + Enter**.

The supplied exercises use local or recorded examples by default. Fresh model calls and GPU training are separate, optional activities.

Windows: activate with `.venv\Scripts\activate`.

One financial language-model workflow across the day

- Choose a model and give it the evidence it is allowed to use.
- Compare base, retrieval-augmented and adapted configurations.
- Test what changes, what remains unsupported and who may act.
- Choose one workflow to trial, with agreed evidence checks, permitted actions and a named reviewer.

STAC AI BOOTCAMP NYC



Financial language-model architectures

Matthew Dixon, PhD, FRM
BeliefLens

21 October 2026 | 9:30–10:15

matthew.dixon@belieflens.org

What Section 1 adds

Which component does what in a financial language-model workflow?

- **Covered in the opening:** a plausible response still needs checking.
- **Section 1:** choose the model, context, tools and control flow.
- **Section 2:** find, extract and supply the required information.

Why go deeper in Section 2? The right architecture can still use the wrong source or number.

Python notebooks: `01_architectures.ipynb`; companion `01b_harbor_tools_mcp_permissions.ipynb`.

Five things you will be able to explain

- What the model receives and what it returns.
- How model weights differ from runtime context.
- How fixed RAG and agentic RAG organise the same capabilities.
- Where retrieval, a calculator and application controls fit.
- When to retrieve current information or consider further training.

Start with the language-model boundary — 1 of 3

1. The model receives an input	Instructions, the question and any supplied evidence are assembled into what the model can read.

Why this matters: A prompt gives instructions; application integration makes tool execution possible.

Start with the language-model boundary — 2 of 3

1. The model receives an input	Instructions, the question and any supplied evidence are assembled into what the model can read.
2. The model can request a tool	A request names a tool and its arguments. The model has not executed the function merely by requesting it.

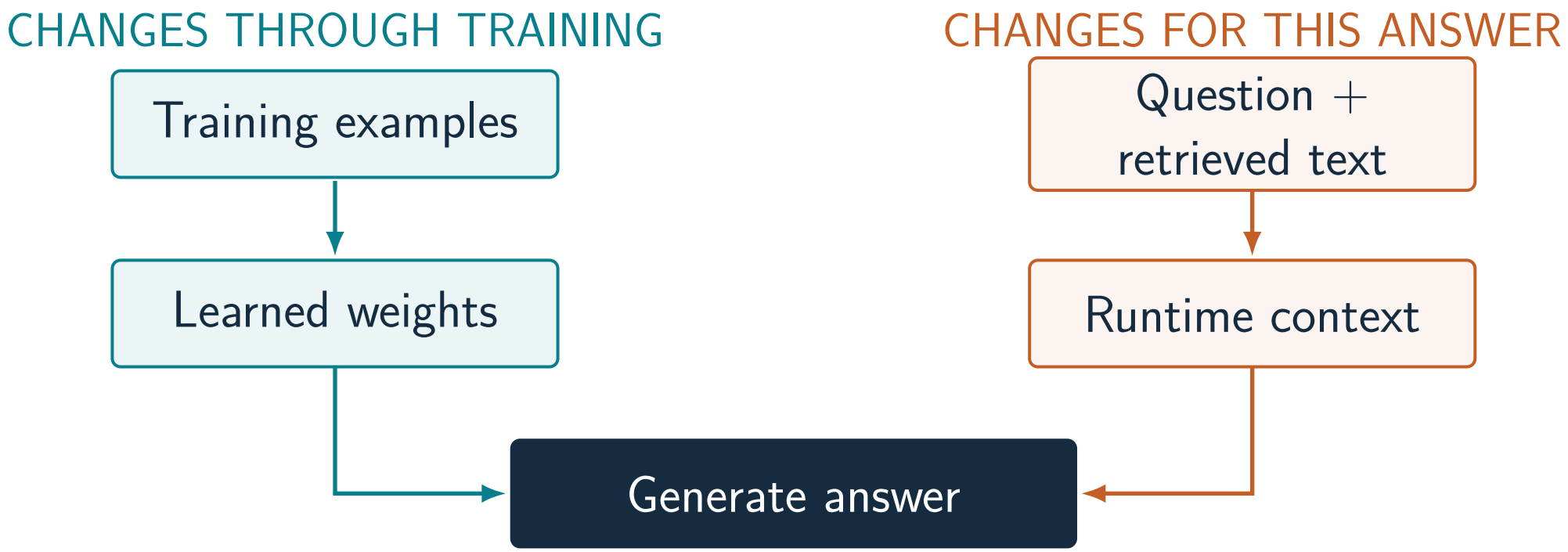
Why this matters: A prompt gives instructions; application integration makes tool execution possible.

Start with the language-model boundary — 3 of 3

1. The model receives an input	Instructions, the question and any supplied evidence are assembled into what the model can read.
2. The model can request a tool	A request names a tool and its arguments. The model has not executed the function merely by requesting it.
3. The application executes permitted requests	Python or a connected service runs the function and returns its result. The model then continues from that result.

Why this matters: A prompt gives instructions; application integration makes tool execution possible.

Two ways information reaches a model



Example: retrieve a newly signed contract today; train only when you want to change learned behaviour.

Different models can do different jobs — 1 of 3

1. Generator: write or request	A generative language model, such as GPT or Qwen, produces an answer or a structured tool request.

Why this matters: One application can use all three. That alone does not make its workflow agentic.

Different models can do different jobs — 2 of 3

1. Generator: write or request	A generative language model, such as GPT or Qwen, produces an answer or a structured tool request.
2. Embedding model: help find candidates	It converts a query and passages into vectors that a search service compares. OpenAI embedding models are one example.

Why this matters: One application can use all three. That alone does not make its workflow agentic.

Different models can do different jobs — 3 of 3

1. Generator: write or request	A generative language model, such as GPT or Qwen, produces an answer or a structured tool request.
2. Embedding model: help find candidates	It converts a query and passages into vectors that a search service compares. OpenAI embedding models are one example.
3. Reranker: compare the shortlist	A cross-encoder, such as a BGE reranker, scores each query–passage pair. It reorders candidates before generation.

Why this matters: One application can use all three. That alone does not make its workflow agentic.

Separate three architecture choices

Choice	Options	Question to ask
Model	General model or specialised model; smaller or larger.	What capability does this task need?
Hosting	Managed API or controlled deployment.	Who operates it and what policies apply?
Control flow	Fixed steps or agent-selected steps.	Which choices should the model make?

A small model can be hosted by a provider. An open-weight model can run in the cloud.

These are separate decisions, not mutually exclusive categories.

Choose the operating limitations first

Deployment	Main benefit	Main responsibility
Managed API	Provider operates serving and infrastructure.	Check service terms, data-policy compatibility and model changes.
Controlled deployment	Pin weights and manage the runtime.	Operate hardware, access controls, updates and monitoring.
Compact model for a bounded task	Potentially lower memory and latency.	Test task coverage and unfamiliar-case handling.

Also compare: cost, latency, reliability, context length, hardware and operating skills.

Example: a new contract motivates retrieval

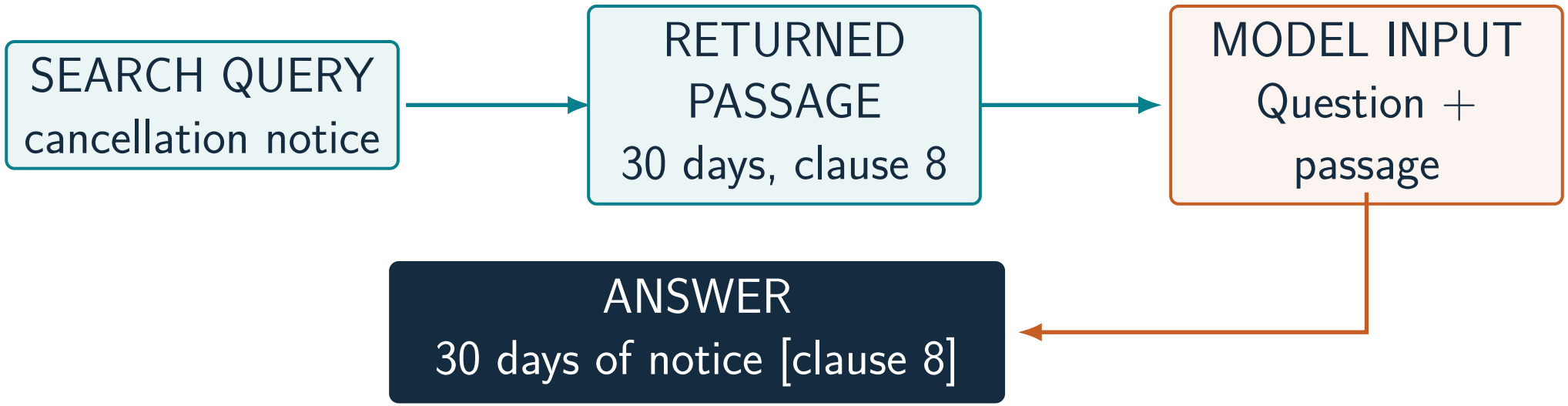


The learned weights may not contain todayś contract.

Practical response: retrieve the current contract and supply its text with the question.

Further training is a separate process for changing learned behaviour.

RAG makes a source part of the model input



The model receives selected text,
not automatic access to the whole library.

See the actual context supplied to the model

Instruction

Answer using the supplied passage. Cite its source.

Question

What is the cancellation notice period?

Retrieved context

[Contract v2, clause 8] Cancellation requires 30 days of written notice.

Generated answer: 30 days of written notice [Contract v2, clause 8].

RAG is not the same as vector search — 1 of 3

1. Retrieval finds information	A search tool can return text, table rows or API results. The application chooses and connects the permitted sources.

Why this matters: A SQL lookup followed by a generated explanation can also be retrieval-augmented generation.

RAG is not the same as vector search — 2 of 3

1. Retrieval finds information	A search tool can return text, table rows or API results. The application chooses and connects the permitted sources.
2. Augmentation puts results in context	The application supplies selected results with the original question. Being in a database is not the same as reaching the model.

Why this matters: A SQL lookup followed by a generated explanation can also be retrieval-augmented generation.

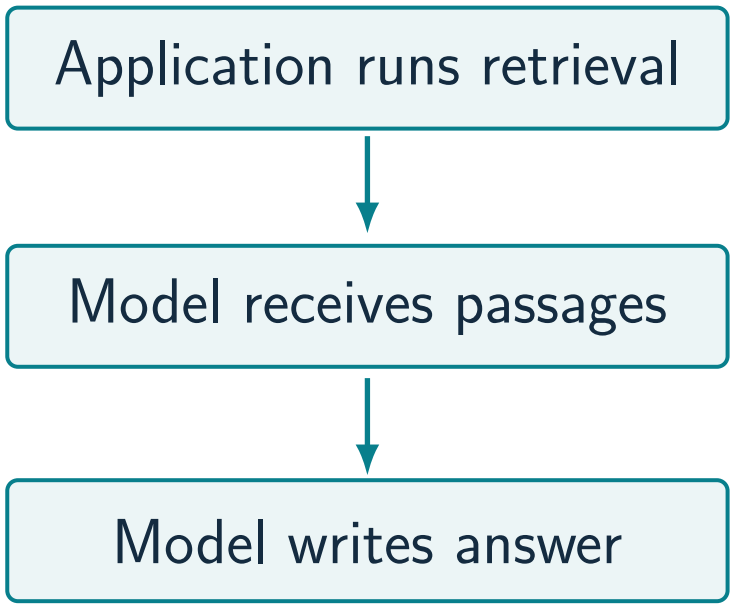
RAG is not the same as vector search — 3 of 3

1. Retrieval finds information	A search tool can return text, table rows or API results. The application chooses and connects the permitted sources.
2. Augmentation puts results in context	The application supplies selected results with the original question. Being in a database is not the same as reaching the model.
3. Generation produces the answer	The language model interprets the supplied context. Vector search is one possible retrieval method, not the whole RAG workflow.

Why this matters: A SQL lookup followed by a generated explanation can also be retrieval-augmented generation.

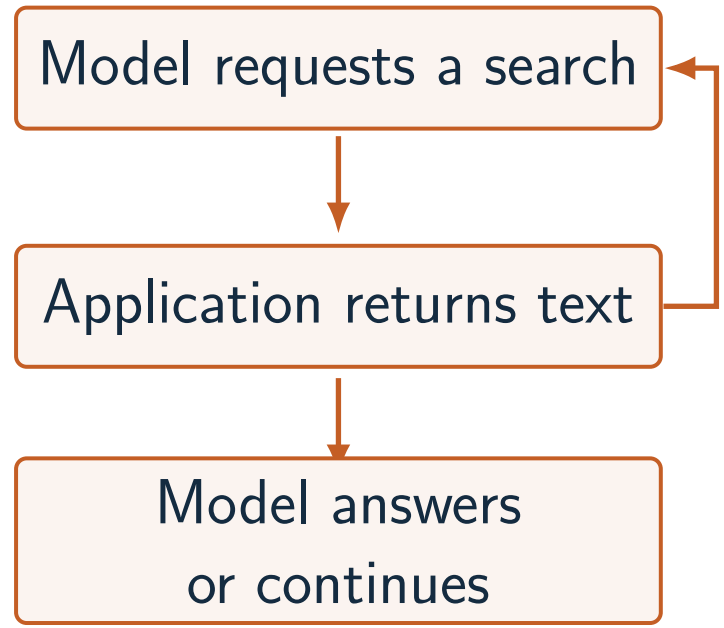
Fixed RAG versus agentic RAG

FIXED RAG



Code fixes the sequence.

AGENTIC RAG



The result can trigger another search.

Which component chooses the next step? — 1 of 3

1. Fixed sequence: code chooses	The application always retrieves, calculates and then asks the model to interpret. The model does not choose those earlier steps.

Why this matters: Inspect the trace: a recommendation to search does not execute a search.

Which component chooses the next step? — 2 of 3

1. Fixed sequence: code chooses	The application always retrieves, calculates and then asks the model to interpret. The model does not choose those earlier steps.
2. Agent loop: the model requests	The model sees a result and can request a follow-up search or calculation. The application executes each permitted request.

Why this matters: Inspect the trace: a recommendation to search does not execute a search.

Which component chooses the next step? — 3 of 3

1. Fixed sequence: code chooses	The application always retrieves, calculates and then asks the model to interpret. The model does not choose those earlier steps.
2. Agent loop: the model requests	The model sees a result and can request a follow-up search or calculation. The application executes each permitted request.
3. Either can follow an annex reference	Code can include a programmed branch. The choice is prescribed branching versus model-selected next steps, not tools versus no tools.

Why this matters: Inspect the trace: a recommendation to search does not execute a search.

Question: is this workflow agentic?

Code always retrieves a contract, adds the passage to the prompt, and asks the model to answer. What controls the steps?

- A. Application code controls a fixed RAG workflow.
- B. The model controls an agentic workflow because it writes the answer.
- C. The search index controls the application.

Answer: the application controls this sequence

Code always retrieves a contract, adds the passage to the prompt, and asks the model to answer. What controls the steps?

A. Application code controls a fixed RAG workflow.

The model generates the answer, but code chose the retrieval step and its place in the sequence.

An agentic version would let the model request additional permitted steps after observing results.

Harbor Metals: introduce the credit question

Fictional commodity company. Can its covenant headroom be called comfortable?

Financial filing	Loan agreement and annex
Debt: $75m; total cash: $15m.	Ceiling: 3.5 times net leverage.
EBITDA: $20m.	Exclude restricted cash identified in Annex A.
Downside EBITDA scenario: $17m.	Annex A: $5m restricted cash.

The filing supplies amounts; the contract defines which cash is eligible.

The retrieval output must reach the model

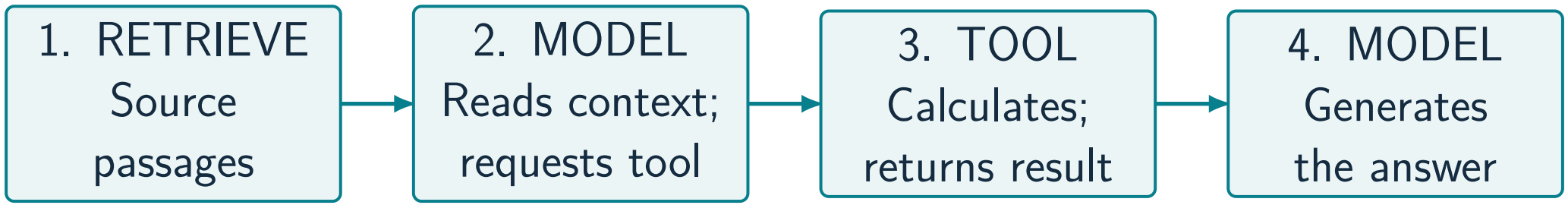
Retrieved source	Passage or record supplied in context
Filing	Debt 75; cash 15; EBITDA 20. Amounts are in millions.
Covenant	Net debt excludes restricted cash. Refer to Annex A.
Annex A	Restricted cash is 5 million.

Derived input: eligible cash $= 15 - 5 = 10$.

A document being in the library does not mean its passage was returned or supplied to the model.

Where retrieval and calculation fit

RAG + calculator: the application supplies source text to the model.



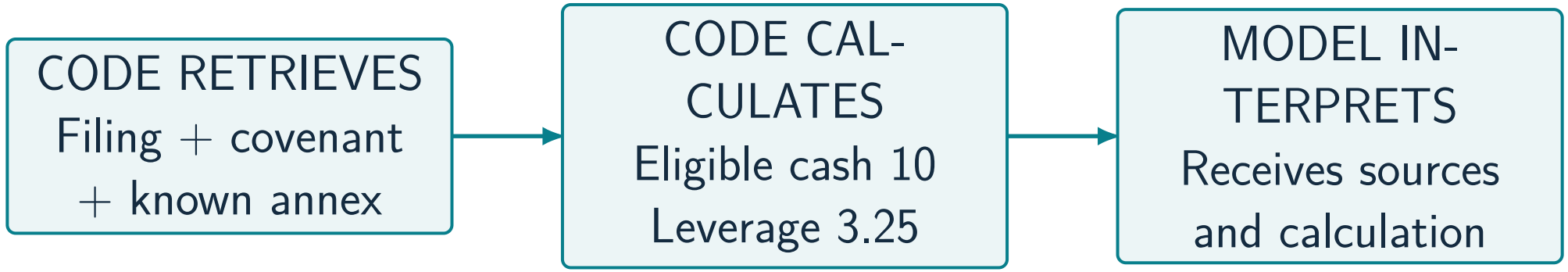
Source text: debt 75; eligible cash 10; EBITDA 20.

```
calculate_leverage(debt=75, eligible_cash=10, ebitda=20)
```

Returned result: 3.25×. The model uses it in its answer.

RAG names the overall retrieve-and-answer pattern, not a separate component.

Fixed workflow: code specifies the required steps



Model input: the credit question + selected passages + the calculator result.

The model need not choose a search query or request a calculator call in this workflow.

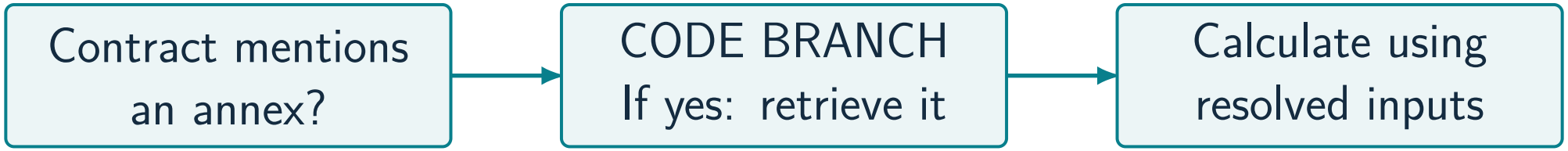
Agent workflow: the result changes the next request

1. Model requests the filing and covenant.
2. Returned clause says: Exclude restricted cash; see Annex A.
3. Model requests Annex A, then calculates with eligible cash 10.

The extra request follows from an observation.

Application code still checks, executes and records every permitted call.

Fixed code can follow a programmed branch too



Same evidence and calculation, different control.

An agent can choose a new request; fixed code can implement known conditional paths.

Neither approach can establish an unavailable annexś contents.

Harbor Metals: connect the model to two functions

Application exposes tools and their argument definitions.

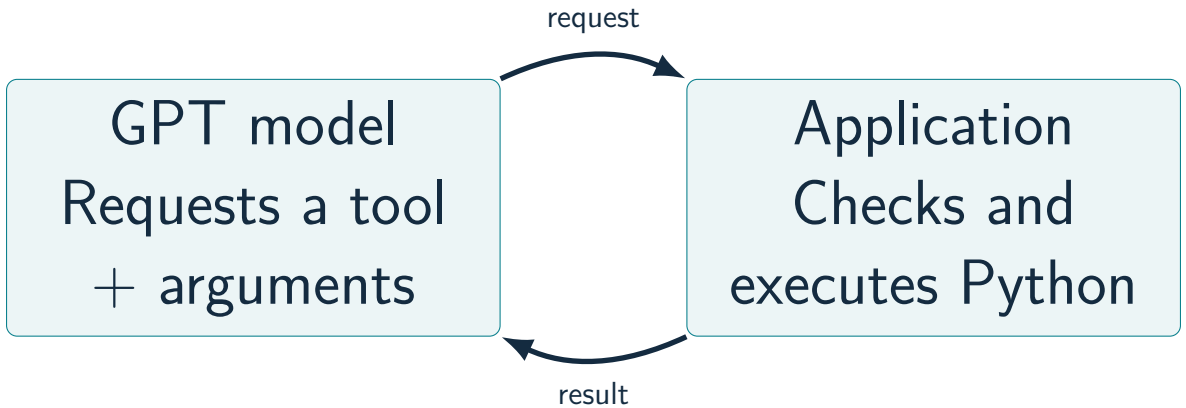
`search_documents(query)`
Returns cited filing, covenant or annex passages.

`calculate_leverage(debt, eligible_cash, ebitda)`
Returns the independently calculated leverage ratio.

Prompt: Assess covenant headroom. Follow annex references; use the calculator; cite sources.

The prompt gives instructions. Application integration makes tools available.

Direct function calling: request, execute, return

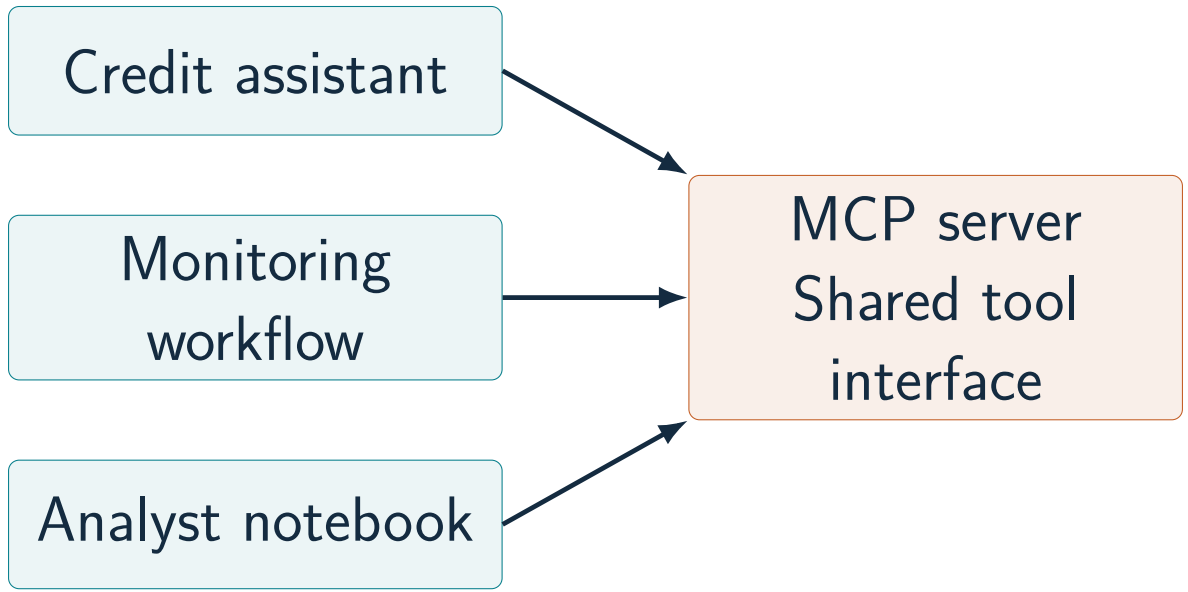


Illustrative trace: search filing → find annex reference → search annex → calculate → answer.

Eligible cash = 15 – 5 = 10; leverage = (75 – 10)/20 = 3.25×.

No MCP server is required for this connection.

When MCP helps: reuse Harbor Metals tools



Same search and calculator functions; different workflows.

MCP standardizes tool discovery and calls. It does not choose the workflow or grant authority.

Permission to calculate is not permission to release

Role	Search / calculate	Release brief
Research	Allowed	Denied
Reviewer	Allowed	Allowed while grant is valid
Revoked reviewer	Allowed	Denied

Enforcement: check the authenticated identity and current grant before execution. Record the result.

A prompt saying “do not release” is not a permission boundary.

Later: identity, revocation and accountable approval.

Class exercise: Harbor Metals tools and permissions

Open: `01b_harbor_tools_mcp_permissions.ipynb`

- Run C01–C05: search, follow ANNEX_A, calculate 3.25×.
- Leave RUN_LIVE and RUN_MCP False for the default path.
- Run C08: research denied; reviewer permitted; revoked reviewer denied.
- Inspect the AUDIT record and confirm only one simulated release.

Explain: connectivity, control flow and permission are separate.

Retrieval and training solve different problems — 1 of 3

1. A required fact is missing	Obtain the current source and supply it as context. Training on old examples cannot establish what an unavailable new document says.

Why this matters: First diagnose missing information versus recurring interpretation failure.

Retrieval and training solve different problems — 2 of 3

1. A required fact is missing	Obtain the current source and supply it as context. Training on old examples cannot establish what an unavailable new document says.
2. The source is present but repeatedly misread	Improve instructions. If errors persist, consider fine-tuning on reviewed demonstrations of the required behavior.

Why this matters: First diagnose missing information versus recurring interpretation failure.

Retrieval and training solve different problems — 3 of 3

1. A required fact is missing	Obtain the current source and supply it as context. Training on old examples cannot establish what an unavailable new document says.
2. The source is present but repeatedly misread	Improve instructions. If errors persist, consider fine-tuning on reviewed demonstrations of the required behavior.
3. An update appears to help	Evaluate on separate unseen cases. Better performance on the training examples alone does not establish generalization.

Why this matters: First diagnose missing information versus recurring interpretation failure.

Class exercise: inspect the handoffs

Open: `01_architectures.ipynb`

- Run S1, A1 and B1. Inspect returned sources and the cash argument 15.
- Run C1. Follow each requested tool and the eligible-cash argument 10.
- Run D1. The fixed annex branch also returns 3.25×.
- Remove ANNEX_A, rerun C1, then restore S1 and compare.

Explain: who selected each step, which evidence was supplied, and which calculation actually ran.

Question: what should change first?

An agent answers from an obsolete agreement. What should you investigate first?

- A. Which source version retrieval selected and supplied.
- B. Whether a larger model gives a different interpretation of that same obsolete text.
- C. Whether preference training makes the response sound more qualified.

Answer: inspect the selected source version

An agent answers from an obsolete agreement. What should you investigate first?

A. Which source version retrieval selected and supplied.

Check the returned source and the context sent to the model.

Changing the model does not establish which agreement is operative.

Section 2 examines these retrieval and source-selection failures.

Architecture choices involve practical trade-offs — 1 of 3

1. Check operating constraints	Data policy, security, permitted processing locations and retention requirements may rule out an otherwise capable service.

Why this matters: Compare designs on the same tasks, including failure handling and operational work.

Architecture choices involve practical trade-offs — 2 of 3

1. Check operating constraints	Data policy, security, permitted processing locations and retention requirements may rule out an otherwise capable service.
2. Measure the complete workload	Count provider calls, searches, calculations, retries and latency. Token price alone is not the cost of a completed task.

Why this matters: Compare designs on the same tasks, including failure handling and operational work.

Architecture choices involve practical trade-offs — 3 of 3

1. Check operating constraints	Data policy, security, permitted processing locations and retention requirements may rule out an otherwise capable service.
2. Measure the complete workload	Count provider calls, searches, calculations, retries and latency. Token price alone is not the cost of a completed task.
3. Match resources and skills	Local hosting needs hardware and operational support. Choose the simplest adequate workflow that meets the measured requirements.

Why this matters: Compare designs on the same tasks, including failure handling and operational work.

Section 1: objectives revisited

- **Model boundary:** identify the model input and output.
- **Information:** distinguish learned weights from supplied context.
- **Control:** explain which component selects and executes each step.
- **RAG:** explain how retrieved text or records reach generation.
- **Next:** Section 2 examines how the information is found and extracted.